Software plays a major role

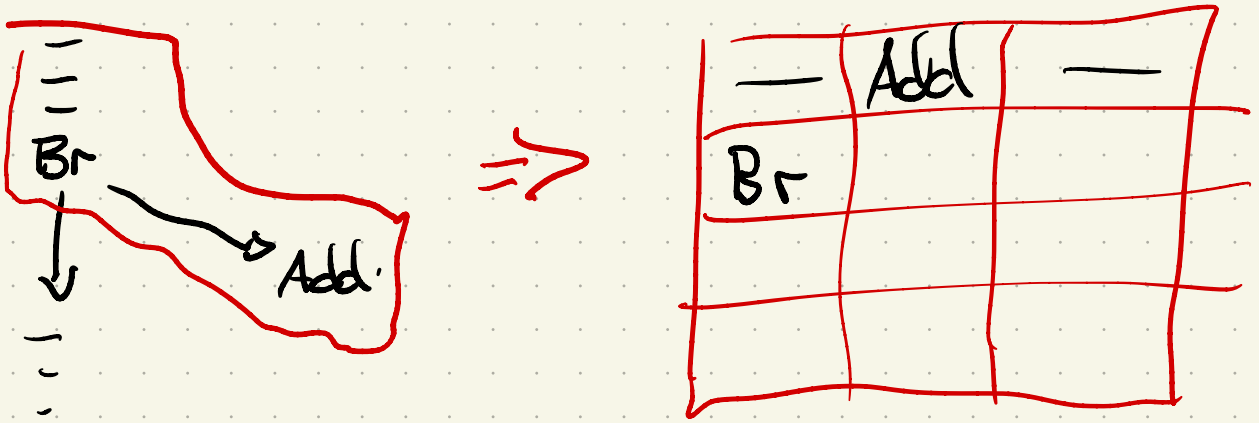
Branch prediction?

Using profile runs

Out-of-order

Software schedules ops out of order

Speculative execution



Anti-/output dependencies?

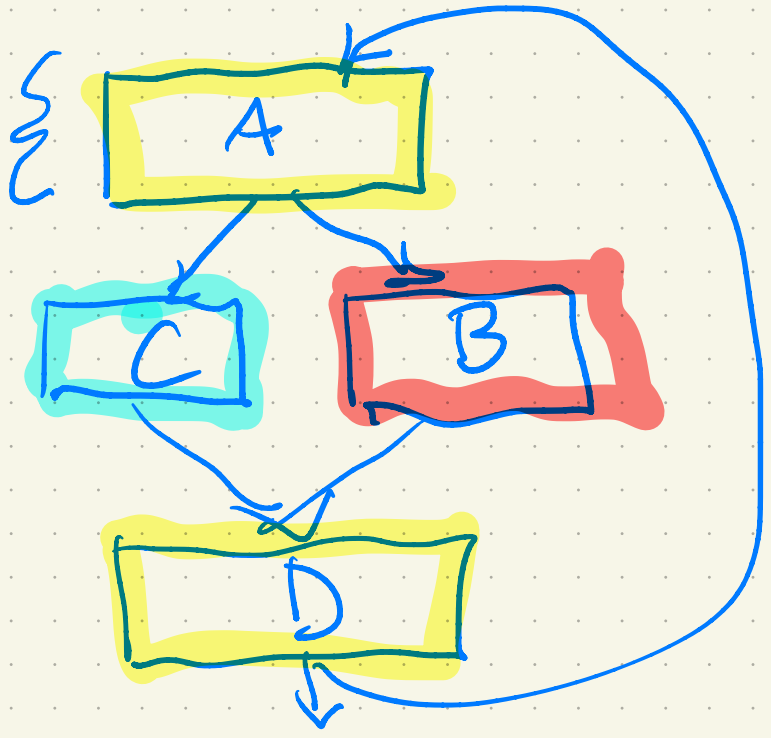
Rename in software ⇒

Very large register file

```
do {
  if A[R5] > 0
      A[R5] = 0;
  else A[R5] = A[R5]*2;
  R5--
} while(R5 > 0);
```

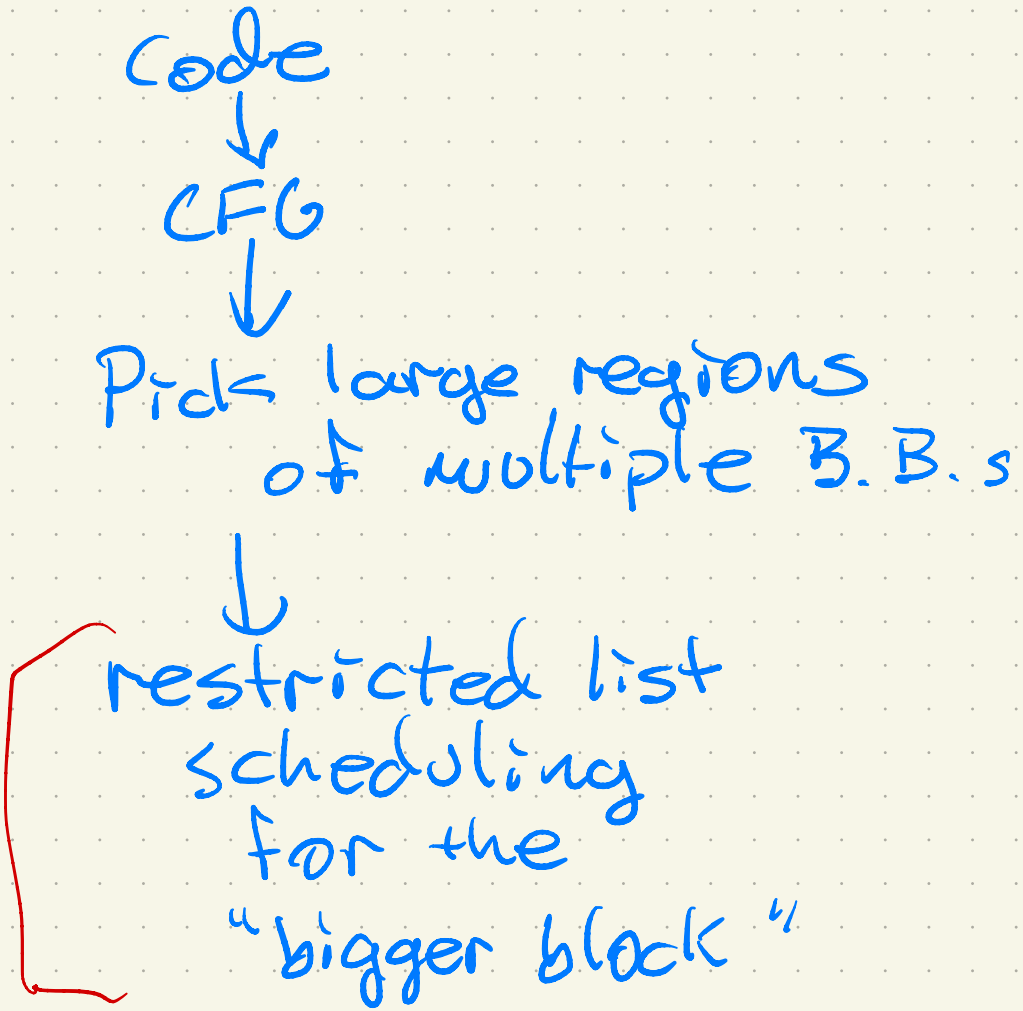
```
A  Loop: lw R4, A(R5)
         Brgt R4, 0, Then
B  Else: Mul R4, R4, 2
         sw R4, A[R5]
         Br After
C  Then: sw 0, A(R5)
D  After: Add R5, R5, -1
         Brgt R5, 0, loop
```

Basic Blocks



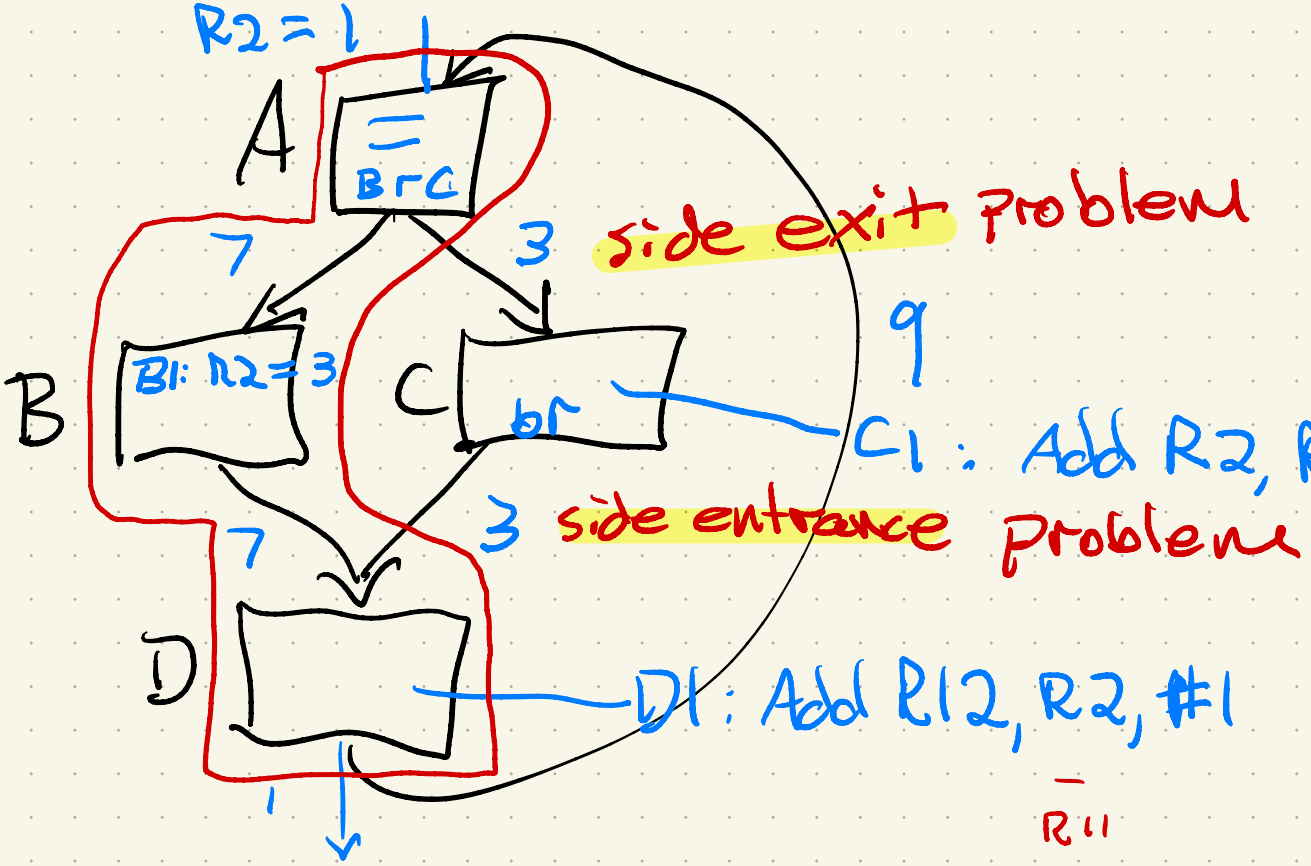
Control Flow Graph CFG

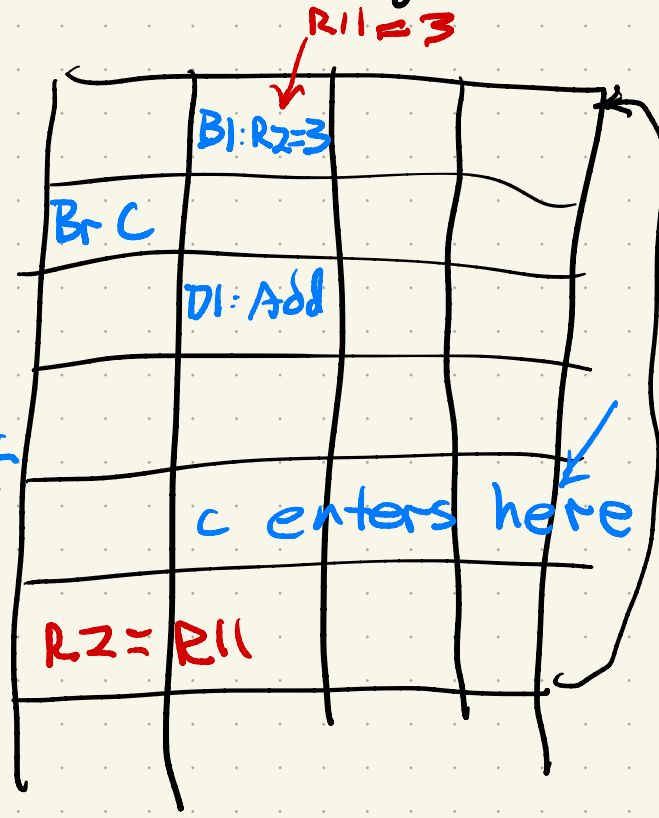
Global instruction scheduling

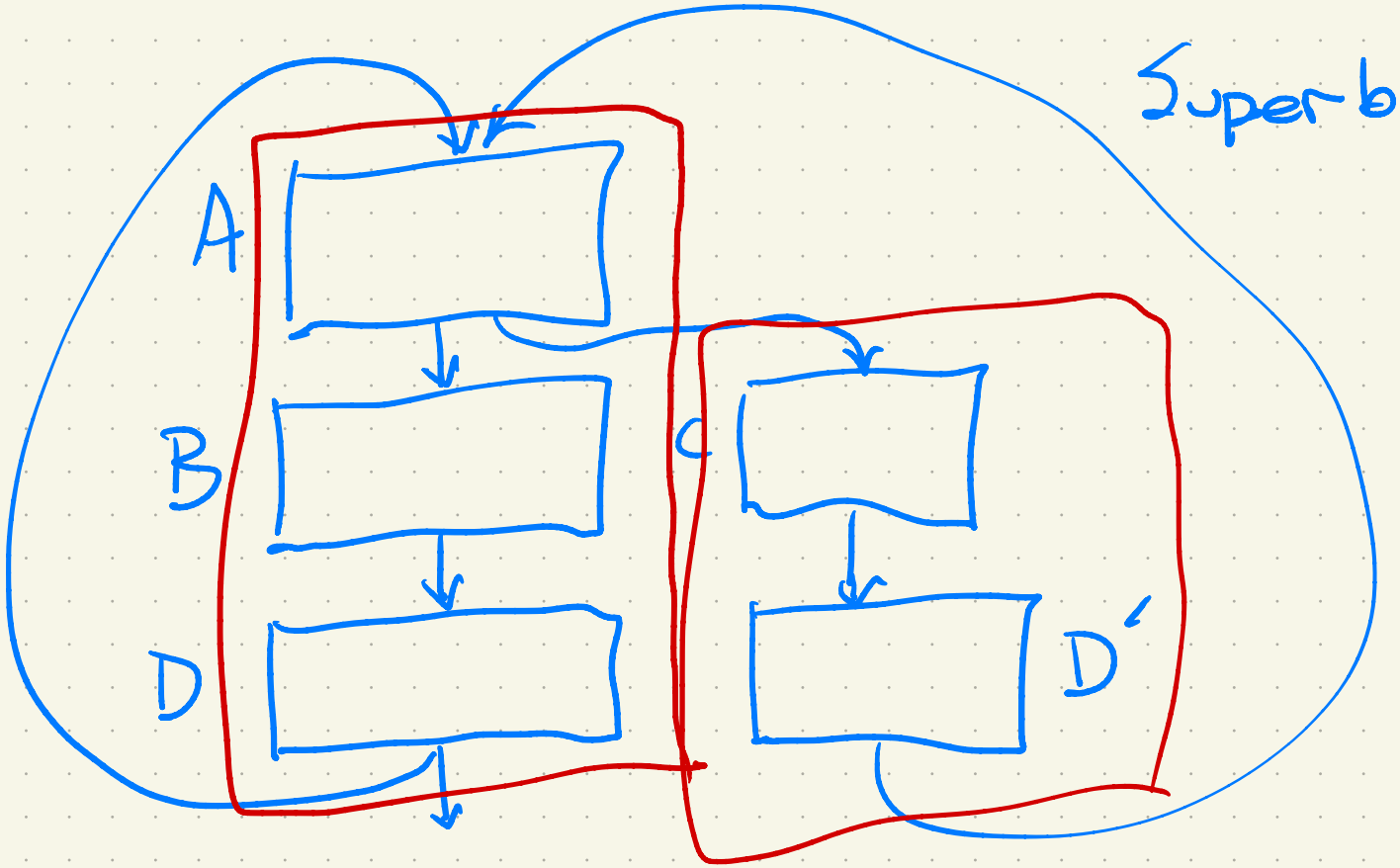


Trace scheduling Fisher/Ellis

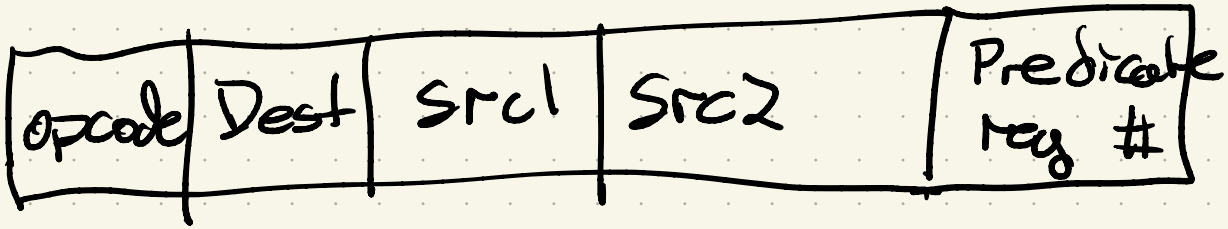
Trace : seq of basic blocks
that tends to execute together







Only side exits now



Predicate registers: 1-bit registers

A Loop: lw R4, A(R5)
Brgt R4, 0, Then

B Else: Mul R4, R4, 2
sw R4, A[R5]
Br After

C Then: sw 0, A(R5)

D After: Add R5, R5, -1
Brgt R5, 0, loop

⇒ "if conversion"

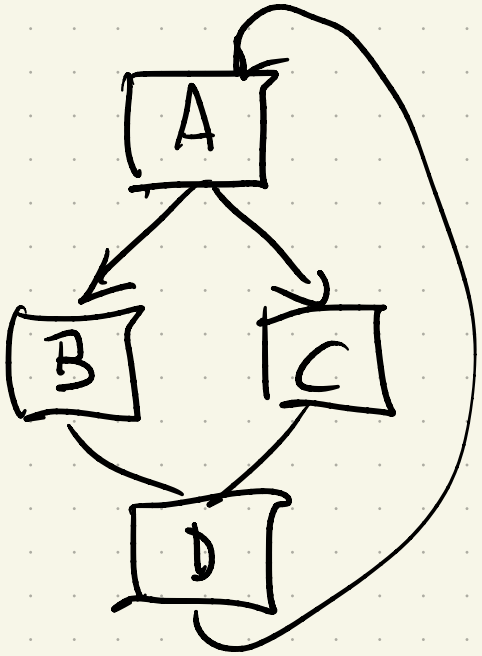
P0 = 1

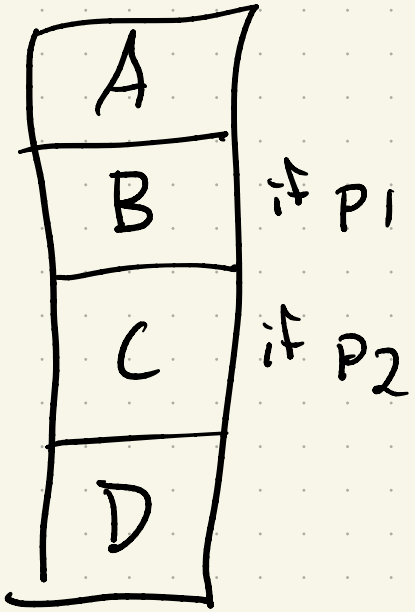
Loop: lw R4, A(R5), P0
CMPP.GT P1, R4, 0, P0
Not P2, P1, P0
Mul R4, R4, 2, P2
sw R4, A(R5), P2
sw 0, A(R5), P1
Add R5, R5, -1, P0
Brgt R5, 0, loop, P0

Predicated execution added to ISA

↓

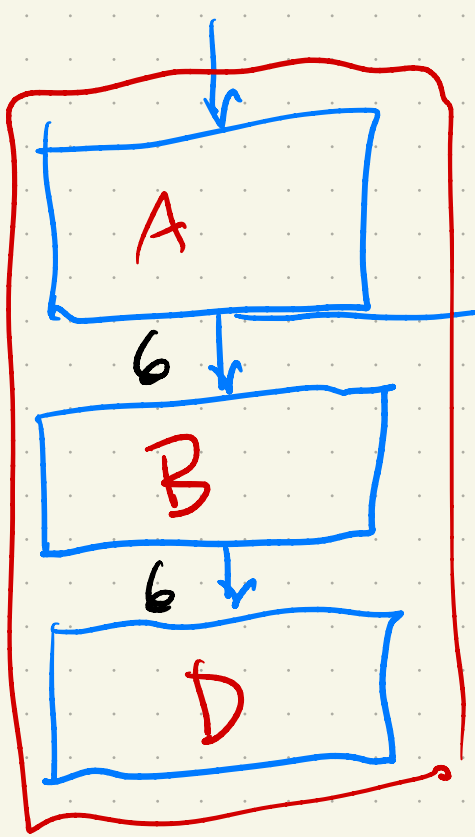
if conversion



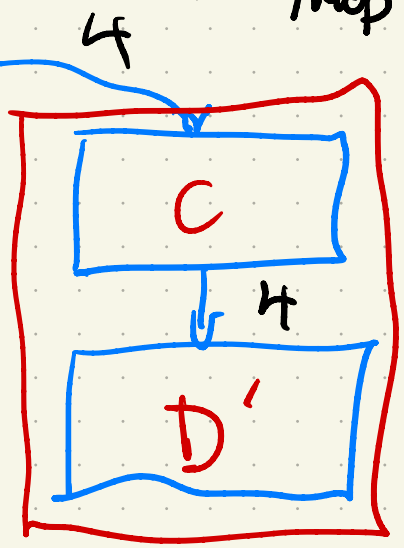


Hyperblock scheduling

A/B/D : 1.1 ops/multiop



C/D : 1 op/mop

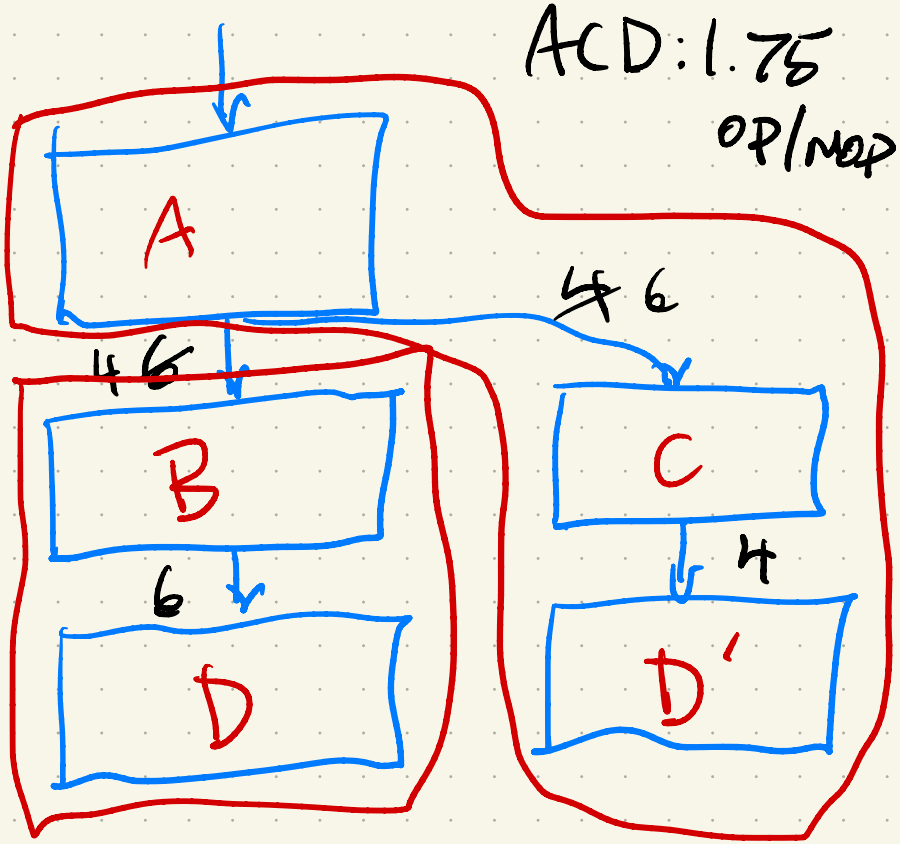


Perf = 6 × 1.1 + 4 × 1

= 1.06

B/D : 1 op/mop

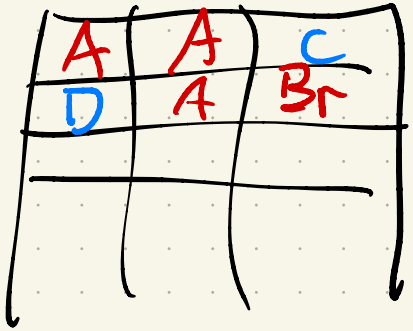
ACD : 1.75 op/mop

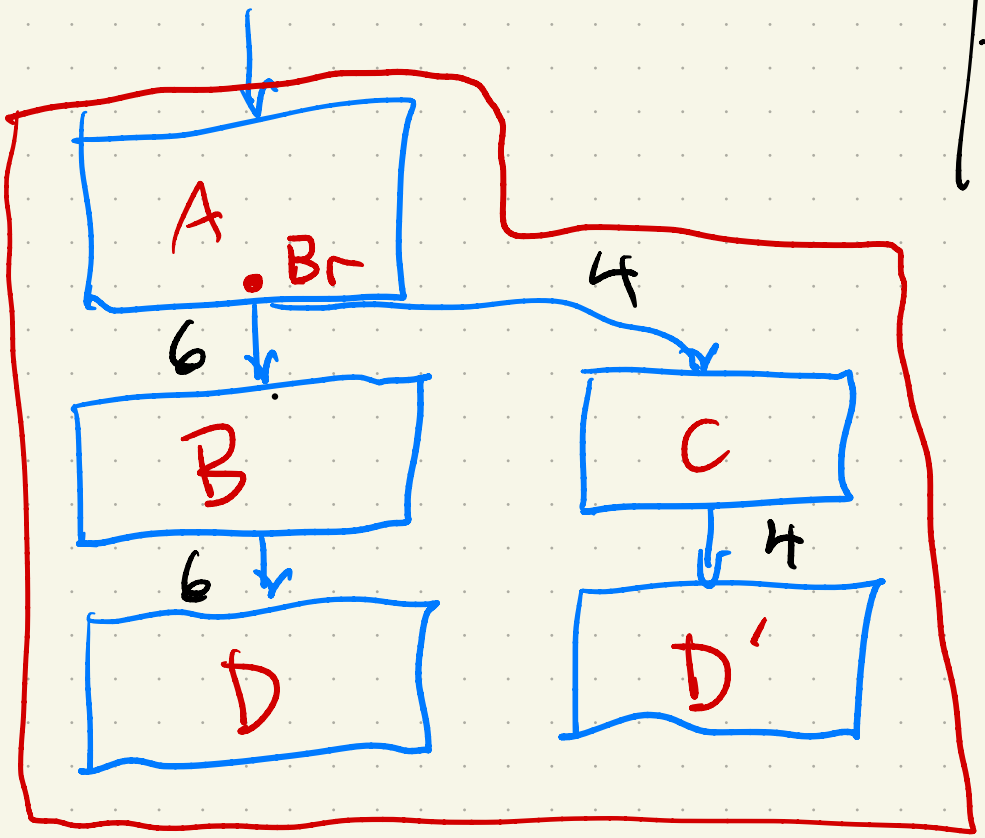


Perf = 6 × 1 + 4 × 1.75

= 1.3

Tree region : Treegion





Instruction scheduling

local (1 BB) vs. Global (across multiple BB's)

Non-loops: Acyclic

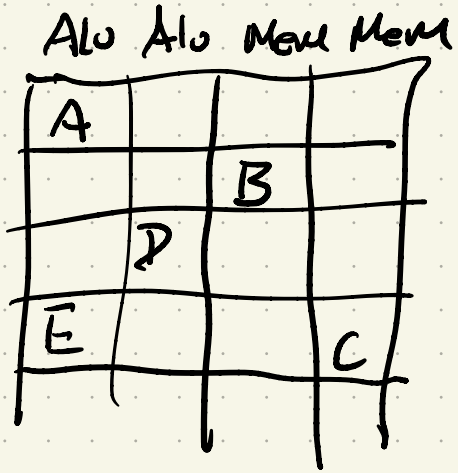
Loops: Cyclic

Cyclic scheduling

```
Loop: A: Add R2,R2,1
      B: Lw R3,0(R2)
      C: Sw R0,0(R3)
      D: Add R10,R10,1
      E: Brgt R10,0,Loop
```

A → B → C

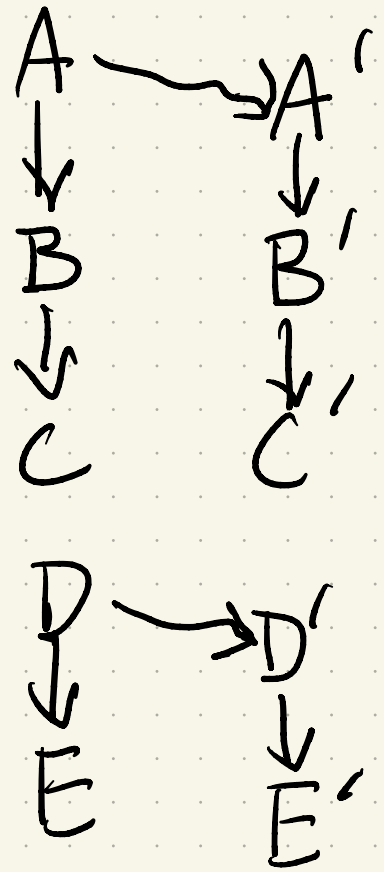
D → E

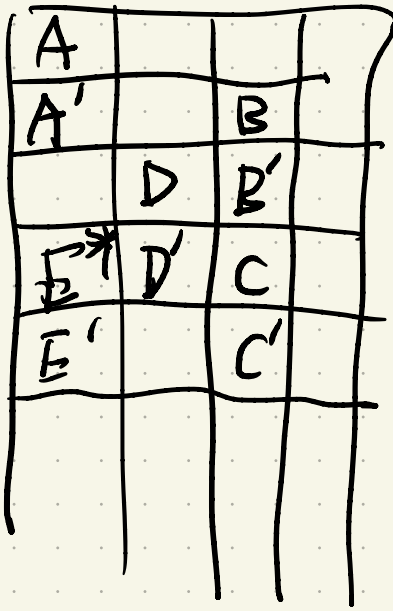


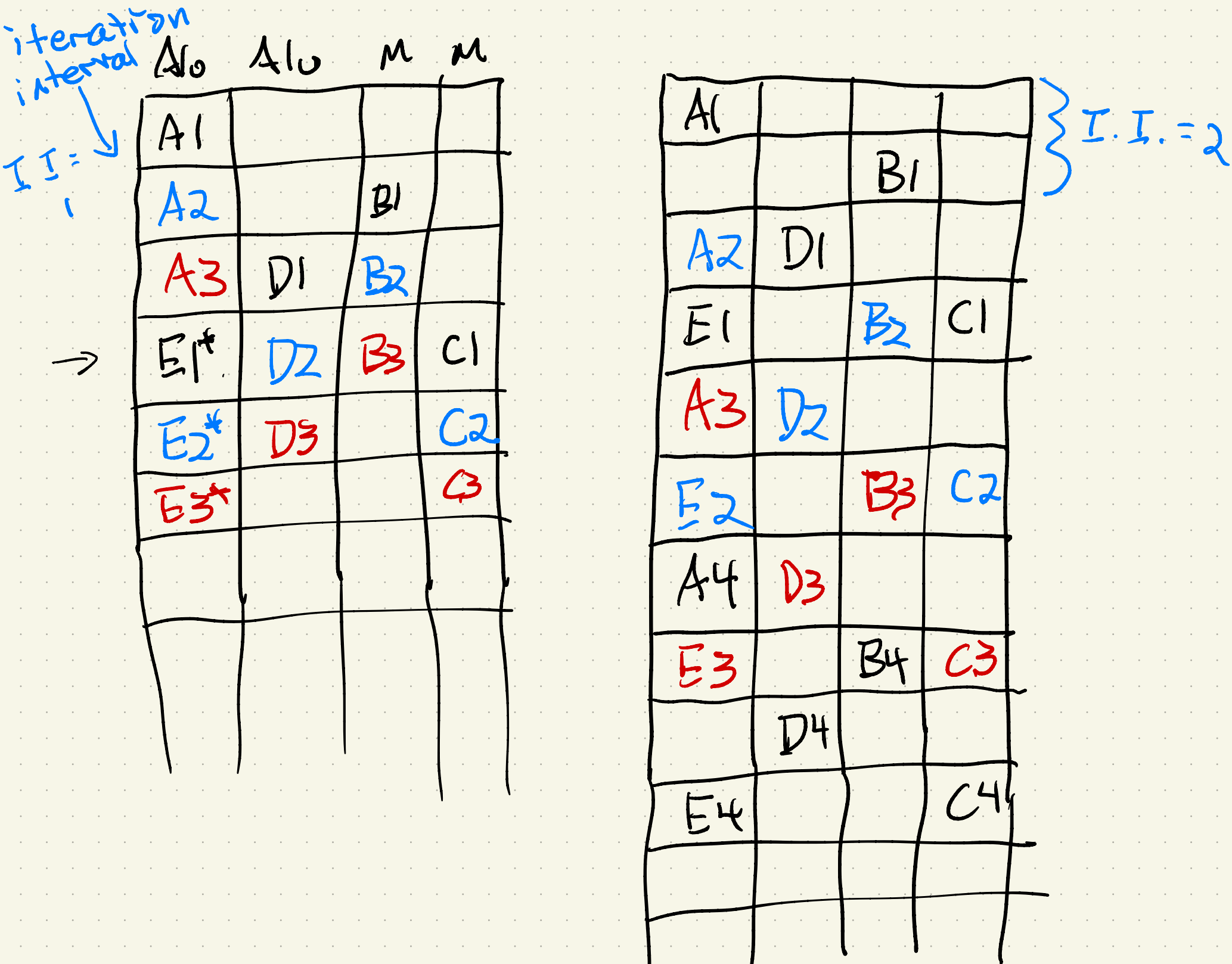
Loop unrolling

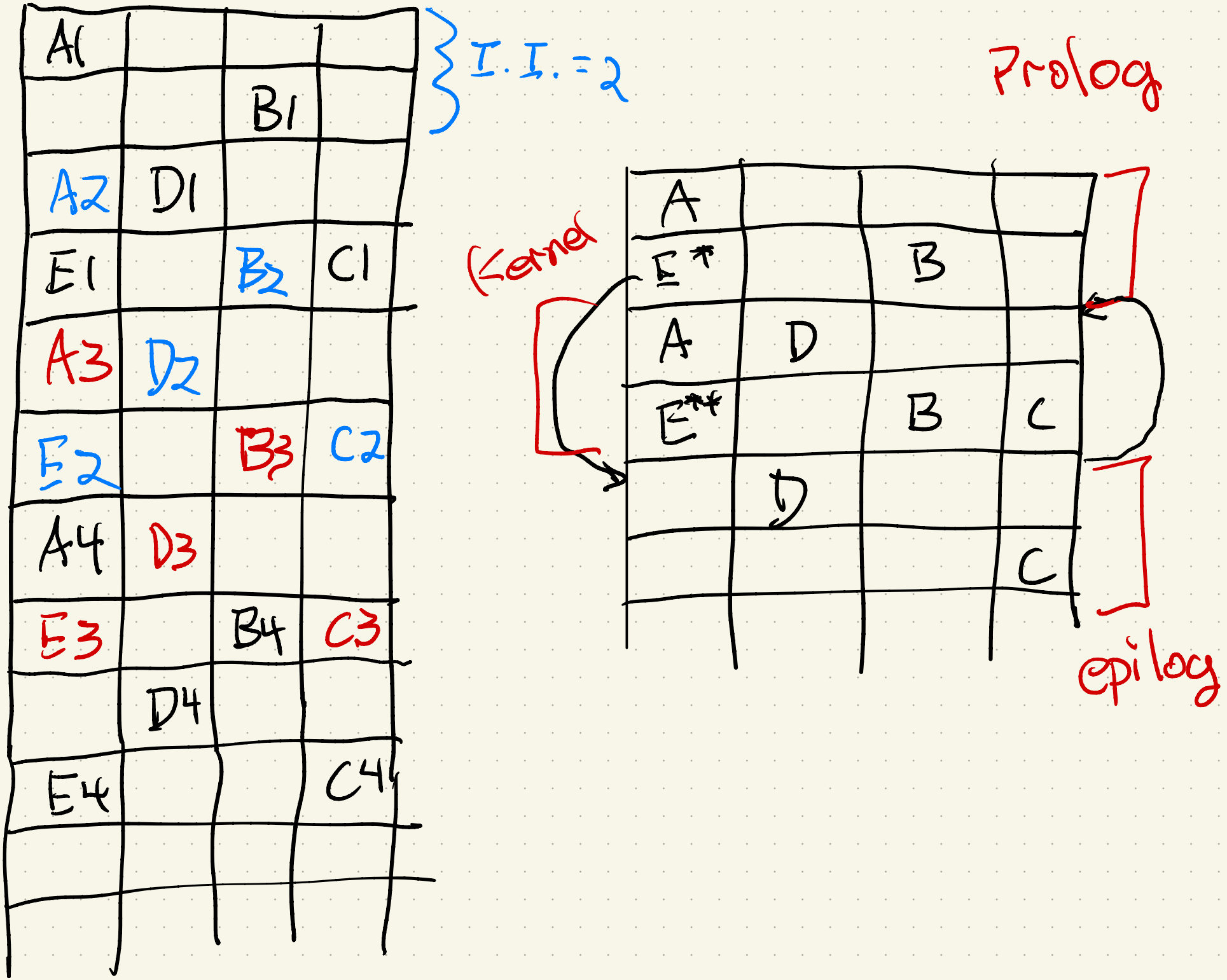
```
Loop: A: Add R2,R2,1
      B: Lw R3,0(R2)
      C: Sw R0,0(R3)
      D: Add R10,R10,1
      E: ~~Brgt R10,0,Loop~~
      E': Brle R10,0,exit

      A': Add R2,R2,1
      B': Lw R3,0(R2)
      C': Sw R0,0(R3)
      D': Add R10,R10,1
      E': Brgt R10,0,Loop
Exit: ...
```






iteration interval
I.I. = 1
ALU
ALU
M
M
A1
A2
A3
E1*
E2*
E3*
B1
D1
B2
D2
B3
D3
C1
C2
C3
A1
B1
A2
D1
E1
B2
C1
A3
D2
E2
B3
C2
A4
D3
E3
B4
C3
D4
E4
C4
I.I. = 2


A1
B1
I.I. = 2
A2
D1
E1
B2
C1
A3
D2
E2
B3
C2
A4
D3
E3
B4
C3
D4
E4
C4
Prolog
A
E*
B
A
D
E**
B
C
D
C
Kernel
epilog